

CS 61A Midterm 3 Study Guide

Trees

```
from __future__ import annotations # don't worry about this
from dataclasses import dataclass, field

@dataclass
class Tree[T]:
    """A Tree has a label (of type T) and a list of branches, which are trees."""
    label: T
    branches: List[Tree[T]] = field(default_factory=list) # branches defaults to []

    def __str__(self):
        return format_tree(self)

def format_tree(t: Tree, indent='') -> str:
    "Format a tree with each branch indented below its label."
    assert isinstance(t, Tree), f'{t!r} is not a Tree'
    assert isinstance(t.branches, list), f'branches of {t!r} is not a list'
    string = indent + str(t.label)
    for b in t.branches:
        string += '\n' + format_tree(b, indent + ' ')
    return string

>>> expr = Tree(10, [Tree(6, [Tree(2), Tree('*'), Tree(3)]), Tree('+'), Tree(4)])
```

- A tree has a root **label** and a list of **branches**; each branch is a tree.
- A tree with no branches is a **leaf**: `t.branches == []`, or not `t.branches`.
- Each location is a **node**; a node is the **parent** of its branches' roots (its **children**).
- A **path** runs from the root down to a node.

```
>>> t = Tree(3, [Tree(1, []),
...             Tree(2, [Tree(1, []), Tree(1, [ ]])])])
>>> t.branches[0]
Tree(label=1, branches=[])
>>> len(t.branches)
2
>>> t.branches[1].label
2
>>> len(t.branches[1].branches)
2
>>> print(expr)
10
  6
  2
  *
  3
  +
  4
```

Tree Recursion

Each branch is a **smaller tree**, so a call on a branch keeps the same promise. Recurse on every branch, then combine.

```
def leaves(t):
    """The leaf values in t.
    >>> leaves(fib_tree(5))
    [1, 0, 1, 0, 1, 1, 0, 1]
    """
    if not t.branches:
        return [t.label]
    else:
        return sum([leaves(b)
                     for b in t.branches], [])

def fib_tree(n):
    if n == 0 or n == 1:
        return Tree(n)
    else:
        left = fib_tree(n-2)
        right = fib_tree(n-1)
        fib_n = left.label + right.label
        return Tree(fib_n, [left, right])
```

repr vs str

The result of calling **repr** on a value is what Python **displays in an interactive session**. The result of calling **str** on a value is what Python **prints** using the **print** function.

```
>>> today = datetime.date(2019, 10, 13)
>>> repr(today) # or today.__repr__()
'datetime.date(2019, 10, 13)'
>>> str(today) # or today.__str__()
'2019-10-13'
```

The result of evaluating an **f-string** literal contains the **str** string of the value of each sub-expression.

```
>>> f'pi starts with {pi}...'
'pi starts with 3.141592653589793...'
>>> print(f'pi starts with {pi}...')
pi starts with 3.141592653589793...
```

@dataclass writes `__repr__`. **Tree / Link** define `__str__`, so `print` uses `format_tree / format_link`.

List Methods (mutate the list)

```
>>> suits = ['coin', 'string', 'myriad']
>>> suits.pop() # remove & return last 'myriad'
>>> suits.remove('string') # first match
>>> suits.append('cup') # add one
>>> suits.extend(['sword', 'club']) # add all
>>> suits[2] = 'spade'
>>> suits
['coin', 'cup', 'spade', 'club']
>>> suits[0:2] = ['diamond'] # replace slice
>>> suits
['diamond', 'spade', 'club']
>>> suits.insert(0, 'heart') # add at index
>>> suits
['heart', 'diamond', 'spade', 'club']
```

List Mutation: Identity, Aliases & Copies

Identity: `<exp0> is <exp1>` is **True** if both evaluate to the **same object**.
Equality: `<exp0> == <exp1>` is **True** if both evaluate to **equal values**.
Identical objects are always equal values.

```
>>> a = [10]
>>> b = a # same list
>>> a == b
True
>>> a.append(20)
>>> a == b
```

```
>>> a = [10]
>>> b = [10] # new list
>>> a == b
True
>>> b.append(20)
>>> a
```

Environment diagram: mutation inside a function

Global frame

- `make_withdraw_list` points to `func make_withdraw_list(balance) [parent=Global]`
- `withdraw` points to `func withdraw(amount) [parent=f1]`

f1: make_withdraw_list [parent=Global]

- `list` points to a list object `[75]` (It changes the contents of the b list)
- `balance` points to `100`
- `withdraw` points to the `withdraw` function object
- `b` points to the `list` object
- `Return value` points to `100`

f2: withdraw [parent=f1]

- `amount` points to `25`
- `Return value` points to `75`

Annotations:

- `withdraw` doesn't reassign any name within the parent
- Name bound outside of `withdraw` def
- Element assignment changes a list

```
def make_withdraw_list(balance: int):
    b = [balance]
    def withdraw(amount: int) -> int:
        if amount > b[0]:
            return 'Insufficient funds'
        b[0] = b[0] - amount
        return b[0]
    return withdraw

withdraw = make_withdraw_list(100)
withdraw(25)
```

Dictionary Mutation

```
True
>>> a
[10, 20]
>>> b # b changed too
[10, 20]
False
>>> a = [10, 20, 30]
>>> list(a) # copy
[10, 20, 30]
>>> a[:] # copy
[10, 20, 30]
```

Dictionary Mutation

```
>>> words["oruguita"] = 'caterpillar'
>>> words["oruguita"] += '\n'
>>> words["oruguita"]
'caterpillar\n'
```

Mutating Trees & Linked Lists

Change a tree or linked list **in place** by assigning to its attributes (`t.label`, `t.branches`, `s.first`, `s.rest`). A mutating function returns **None**; a non-mutating one returns a **new Tree / Link**.

Iterators & Lazy Evaluation

An **iterable** is anything you can loop over (list, tuple, range, dict, str). An **iterator** remembers a **position** in one.

iter(iterable) Return an iterator over the elements of an iterable value
next(iterator) Return the next element of an iterator (StopIteration when none are left)

```
>>> s = [3, 4, 5]
>>> t = iter(s)
>>> next(t)
3
>>> next(t)
4
>>> d = {'one': 1, 'two': 2}
>>> k = iter(d)
>>> next(k)
'one'
>>> next(k)
'two'
>>> v = iter(d.values())
>>> next(v)
1
>>> next(v)
2
```

- An iterator is **one pass**: once it reaches the end it stays empty. Calling **iter** again gives a new one that starts over; the list isn't changed.

Lazy: return an iterator and compute nothing yet.
map filter zip reversed, generators

Eager: look at every element right now.
list tuple sorted sum max min, for loops

Generators

Many built-in Python sequence operations return **iterators that compute results lazily**. To view the contents of an iterator, place the resulting elements into a **container**:

```
map(func, iterable) Iterate over func(x) for x in iterable
filter(func, iterable) Iterate over x in iterable if func(x)
zip(first_iter, second_iter) Iterate over co-indexed (x, y) pairs
reversed(sequence) Iterate over x in a sequence in reverse order
list(iterable) Create a list containing all x in iterable
tuple(iterable) Create a tuple containing all x in iterable
sorted(iterable) Create a sorted list containing x in iterable
```

What "lazy" means: making `map(func, s)` does **not** call `func` at all. Each time **next** asks, it calls `func` on **one** more element. Passing the iterator to something eager (`list`, `sum`, a `for` loop) forces **all the remaining** elements at once. **Why**: you only pay for elements you use.

- Gotchas**: printing a map shows `<map object at ...>`: wrap it in `list`. A map has no `len`. A used-up iterator gives `[]`. `zip` stops at the shorter input.

Generators

A **generator function** is a function that **yields** values instead of **returning**. Calling it returns a **generator** (a lazy iterator) **without running the body**. Each **next** runs the body until the next **yield**, then pauses there.

```
>>> def plus_minus(x):
...     yield x
...     yield -x
>>> t = plus_minus(3)
>>> next(t)
3
>>> next(t)
-3
yield from <iterable> yields every value of the iterable, one by one.
def a_then_b(a, b):
    yield from a
    yield from b
>>> list(a_then_b([3, 4], [5, 6]))
[3, 4, 5, 6]
```

Efficiency: Orders of Growth

Exponential	recursive fib : incrementing <i>n</i> multiplies <i>time</i> by a constant	$\Theta(b^n)$	$O(b^n)$
Quadratic	overlap : incrementing <i>n</i> increases <i>time</i> by <i>n</i> × a constant	$\Theta(n^2)$	$O(n^2)$
Linear	slow exp : incrementing <i>n</i> increases <i>time</i> by a constant	$\Theta(n)$	$O(n)$
Logarithmic	exp_fast : doubling <i>n</i> only increments <i>time</i> by a constant	$\Theta(\log n)$	$O(\log n)$
Constant	increasing <i>n</i> doesn't affect time	$\Theta(1)$	$O(1)$

Python Object System

Idea: All bank accounts have a **balance** and an account **holder**; the **Account** class should add those attributes to each of its **instances**.

```
class Account:
    interest = 0.02 # class attribute
    def __init__(self, account_holder):
        self.balance = 0
        self.holder = account_holder
    def deposit(self, amount):
        self.balance = self.balance + amount
        return self.balance
    def withdraw(self, amount):
        if amount > self.balance:
            return 'Insufficient funds'
        self.balance = self.balance - amount
        return self.balance
```

When a class is called:

- A new instance of that class is created.
- The `__init__` method of the class (the **constructor**) is called with the new object as its first argument (named **self**), along with any additional arguments provided in the call expression.

```
>>> a = Account('Jim')
>>> a.holder
'Jim'
>>> a.balance
0
```

`self` should always be bound to an instance of the `Account` class or a subclass of `Account`

Dot Notation: Attributes vs Methods

`<expression> . <name>`: the `<expression>` can be any valid Python expression; the `<name>` must be a simple name. Evaluates to the value of the attribute looked up by `<name>` in the object that is the value of the `<expression>`.

Expression	Evaluates to	What it means
<code>a.balance</code>	a value	an attribute : no parentheses (<code>a.balance()</code> errors)
<code>a.deposit</code>	a bound method	<code>a</code> is already self ; not called yet
<code>a.deposit(2)</code>	a call	method invocation : object before the dot
<code>Account.deposit</code>	a plain function	looked up on the class: nothing bound
<code>Account.deposit(a, 5)</code>	a call	function call : all args inside, incl. self

```
>>> type(Account.deposit)           >>> Account.deposit(a, 5)
<class 'function'>                  10
>>> type(a.deposit)                  >>> a.deposit(2)
<class 'method'>                    12
```

To evaluate a dot expression:

1. Evaluate the `<expression>` to the left of the dot, which yields the object of the dot expression
2. `<name>` is matched against the instance attributes of that object; if an attribute with that name exists, its value is returned
3. If not, `<name>` is looked up in the class, which yields a class attribute value
4. That value is returned unless it is a function, in which case a **bound method** is returned instead

Class vs. Instance Attributes

Assignment statements with a dot expression on their left-hand side affect attributes for the object of that dot expression:

- If the object is an **instance**, then assignment sets an **instance attribute**
 - If the object is a **class**, then assignment sets a **class attribute**
- ```
>>> jim_account = Account('Jim') >>> jim_account.interest = 0.08
>>> tom_account = Account('Tom') >>> jim_account.interest
>>> tom_account.interest 0.08
0.02 >>> tom_account.interest
>>> jim_account.interest 0.04
0.02 >>> Account.interest = 0.05
>>> Account.interest = 0.04 >>> tom_account.interest
>>> tom_account.interest 0.05
0.04 >>> jim_account.interest
>>> jim_account.interest 0.08
0.04
```

|                                                                                                             |                                                              |                                            |
|-------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|--------------------------------------------|
| Account class attributes<br>interest: <del>0.02</del> <del>0.04</del> 0.05<br>(withdraw, deposit, __init__) | jim_account<br>balance: 0<br>holder: 'Jim'<br>interest: 0.08 | tom_account<br>balance: 0<br>holder: 'Tom' |
|-------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|--------------------------------------------|

Inheritance

```
class CheckingAccount(Account):
 """A bank account that charges for withdrawals."""
 withdraw_fee = 1
 interest = 0.01
 def withdraw(self, amount):
 return Account.withdraw(self, amount + self.withdraw_fee)
 # or
 return super().withdraw(amount + self.withdraw_fee)
• super().withdraw(x): look up withdraw starting in the base class; self is filled in for you.
• Account.withdraw(self, x): get the plain function through the class, so pass self yourself.
```

**Lookup order.** To look up a name in a class: (1) if it names an attribute in the class, return the attribute value; (2) otherwise, look up the name in the **base class**, if there is one. So for `obj.name`: **instance** → **its class** → **base class** → ... → **AttributeError**. Assignment never looks up: it sets the name on the object before the dot.

```
>>> ch = CheckingAccount('Tom') # Calls Account.__init__
>>> ch.interest # Found in CheckingAccount
0.01
>>> ch.deposit(20) # Found in Account
20
>>> ch.withdraw(5) # Found in CheckingAccount
14
```